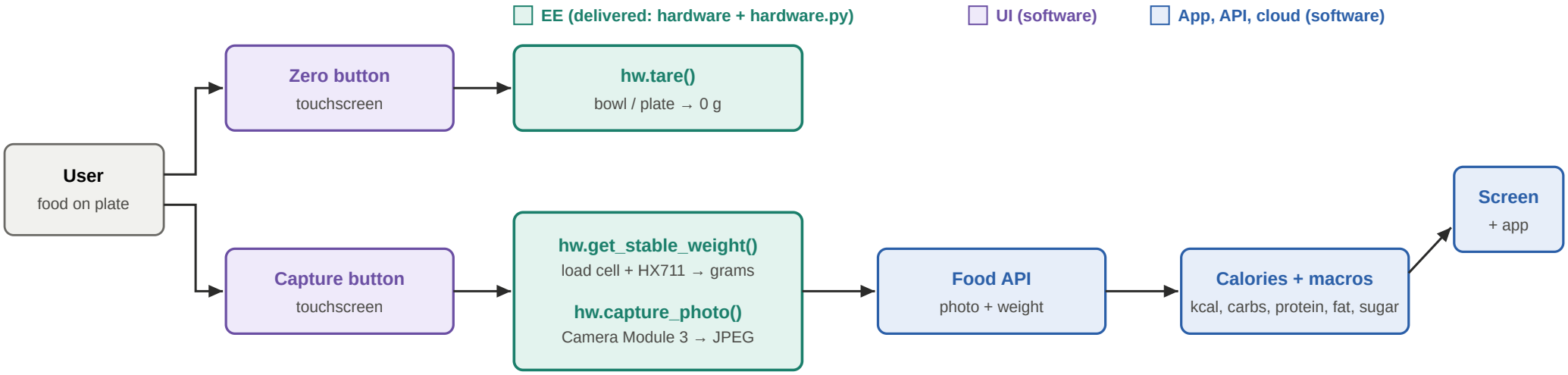
# Smart Food Scale · Hardware Handover

What the electrical side delivers to the software team: a tested Raspberry Pi 4B with load cell, camera and touchscreen, and one Python module that gives the app the weight and the photo.

Prepared 20 October 2026 · Hardware status: built, tested, ready for the app

## How the app uses the hardware




## Delivered

| Item | What it is |
|---|---|
| Hardware | Pi 4B + 5 kg load cell/HX711 + Camera Module 3 + 3.5" touchscreen, on a 3S battery supply |
| hardware.py | The app interface: weight, stable weight, zero, photo, overload |
| scale.py | Load-cell driver + calibration and test tool |
| scale_calibration.json | Calibration of the current stand (200 g reference) |
| This file | Access, connections, interface, rules and test results |



Initial stand with 200 g of reference weights (photo by Camera Module 3)

**The software team owns the buttons, the API call and the result display. The EE side owns everything in green.**

# Access and system setup

How to reach the Pi and what is already configured on it.

## System

| Setting | Value |
|---|---|
| **Board** | Raspberry Pi 4B, 4 GB |
| **OS** | Raspberry Pi OS 64-bit (Debian Trixie), with desktop |
| **Hostname** | `foodscale` |
| **User · password** | `foodscale` · `123456789` |
| **Keyboard · time zone** | us · Asia/Riyadh |
| **SSH** | enabled (password login) |
| **Python libraries** | lgpio (GPIO), picamera2 (camera): both preinstalled |
| **Files** | `/home/foodscale/`: hardware.py, scale.py, scale_calibration.json, photos/ |

## Connect from a laptop

1. Turn on any Wi-Fi named `foodscale-net`, password `123456789`, band **2.4 GHz**: a laptop hotspot (Windows: **Settings → Network → Mobile hotspot**) or a router. The Pi joins any network with this exact name and password automatically.
2. Read the Pi's IP under *Connected devices* (was `192.168.137.166`).
3. PowerShell: `ssh foodscale@192.168.137.166`
4. Copy files to the Pi (from Windows): `scp file.py foodscale@192.168.137.166:~`
5. Copy a photo back: `scp foodscale@192.168.137.166:~/photos/x.jpg .`

### Phone hotspots block it

Most phone hotspots stop devices seeing each other (client isolation): the Pi gets internet but SSH times out. Use the laptop hotspot or a normal router.

## Screen settings · `/boot/firmware/config.txt`

```
dtoverlay=vc4-kms-v3d                # default graphics driver (line 26)
dtoverlay=waveshare_35DSI,35E,dsi1   # 3.5" DSI LCD (E) on the DISPLAY port
```

Driver file `Waveshare_35DSI.dtbo` is in `/boot/firmware/overlays/`. Do not remove either line. Touch (Goodix GT911) works without extra setup.

## Quick health checks

| Command | Healthy result |
|---|---|
| `vcgencmd get_throttled` | `throttled=0x0` (no low voltage) |
| `rpicam-hello --list-cameras` | lists `imx708` |
| `python3 scale.py weigh` | live grams, ±0.1 g wobble |
| `python3 hardware.py` | self-test: zero, stable weight, photo |

**Change the default password before the demo: type passwd on the Pi.**

# Hardware connections

Where each part is connected. Connect or disconnect anything only with the Pi powered off.

## Connections

| Part | Pi connection | Signal / cable | Notes |
|---|---|---|---|
| **Load cell → HX711** | via HX711 board | red E+ · black E− · white A− · green A+ | 5 kg rated, soldered to the HX711 |
| **HX711 VCC** | pin 1 | 3.3 V | not pin 2 (5 V) |
| **HX711 GND** | pin 6 | ground | |
| **HX711 DT** | pin 31 (GPIO6) | data out | read by hardware.py |
| **HX711 SCK** | pin 29 (GPIO5) | clock | must never stay high > 60 µs (handled in the driver) |
| **Camera Module 3** | CAMERA port (CSI) | 15-pin ribbon | silver contacts toward the HDMI ports |
| **3.5" DSI LCD (E)** | DISPLAY port (DSI) | 15-pin ribbon, carries 3.3 V and touch | a loose ribbon gives a black screen |
| **Power** | USB-C | 5.1 V from the battery pack (LM2596 buck) | or a 5 V / 3 A adapter during development |

## Ratings the software must respect

| Quantity | Value |
|---|---|
| Load-cell capacity | 5 kg total on the sensor (plate + bowl + food) |
| Overload warning | 4.5 kg ( `hw.is_overloaded()` ) |
| Load-cell sample rate | ≈ 10 readings/s (HX711, fixed) |
| Camera | 12 MP max (4608 × 2592), autofocus; default photo 2304 × 1296 |
| Screen | 3.5", 640 × 480, capacitive touch |
| Supply current | ≈ 1.65 A at 5 V worst case, all parts on |

### Free GPIO pins

Only pins 1, 6, 29 and 31 are used. Everything else on the 40-pin header is free for the software team (e.g. a physical button or LED).

**Do not use** GPIO5 and GPIO6 for anything else, and do not run another program that reads the HX711 at the same time as the app: only one program may own the load cell.

**Pin numbers are physical header positions; GPIO numbers are what the code uses.**

# Software interface · hardware.py

Everything the app needs from the hardware, in seven calls. The app never touches GPIO pins, timing or calibration.

## Functions

| Call | Returns | Use |
|---|---|---|
| hw.start() | — | once at app start; starts the load-cell thread and zeroes |
| hw.get_weight() | 152.3 (g) | live display; updates ≈ 10×/s |
| hw.get_stable_weight() | 152.3 or None | Capture button; waits until steady (±1 g for 1 s, max 5 s) |
| hw.tare() | True / False | Zero button; bowl or plate becomes 0 g |
| hw.capture_photo() | path to JPEG | Capture button; autofocus, saved in ~/photos/ |
| hw.is_overloaded() | True / False | over 4.5 kg on the sensor |
| hw.stop() | — | at app exit; releases GPIO and camera |

### Send the weight with the photo

The photo tells the API *what* the food is; the weight tells it *how much*. Calories = kcal per gram × grams, so the API needs both.

### Real-time reading

Run the app with `sudo` (or as root via systemd) so the load-cell thread gets real-time priority. Without it everything works, with rare 0.4 s pauses in new readings.

## Example: both buttons

```
# the two screen buttons, any UI framework
import hardware as hw

hw.start()                          # app start-up, plate empty (zeroes)

def on_zero_button():
    if not hw.tare():
        show("Hold still, then press Zero again")

def on_capture_button():
    grams = hw.get_stable_weight()  # waits ≥ 1 s for a steady reading
    if grams is None:
        show("Weight not steady, try again"); return
    if hw.is_overloaded():
        show("Too heavy (max 5 kg)"); return
    photo = hw.capture_photo()      # autofocus + JPEG, ≈ 1 s
    result = call_food_api(photo, grams)    # software team
    show(result)

# live display, e.g. every 200 ms:  label.text = f"{hw.get_weight():.0f} g"
```

**Self-test before writing the app: python3 hardware.py**

# Rules and troubleshooting

What keeps the readings correct, and what to check first when something stops working.

## Rules

1. **Recalibrate after any change to the stand** (plate, screws, spacers): `python3 scale.py calibrate` with the 200 g set, then check 10–200 g.
2. **The plate must touch only the load cell.** Keep a 2–3 mm gap to the frame and keep wires off the plate, or part of the weight is lost.
3. **Start the app with the plate empty**: `hw.start()` zeroes on what it sees.
4. **One program owns the load cell.** Stop scale.py tests before starting the app.
5. **Ribbons only with power off.** Close both latches fully.
6. **Shut down properly** ( `sudo shutdown -h now` ) before switching the battery off, to protect the SD card.

## Troubleshooting

| Symptom | Cause → fix |
|---|---|
| **Black screen** | DSI ribbon not seated → power off, reseat both ends |
| **`HX711 not ready`** | VCC, GND, DT or SCK wire → check pins 1, 6, 31, 29 |
| **Weight stuck at a limit** | load-cell wire loose (E+, E−, A+, A−) |
| **Weight drifts or jumps** | plate touching the frame, or a wire on the plate |
| **Weight wrong by a fixed %** | stand changed → recalibrate |
| **`get_stable_weight()` = None** | food still moving, or the stand vibrates |
| **Camera not found** | CSI ribbon → `rpicam-hello --list-cameras` |
| **Blurry photo** | object closer than ≈ 10 cm, or no light |
| **Pi resets / lightning icon** | low battery → `vcgencmd get_throttled`, charge |
| **SSH times out** | phone hotspot isolation → use laptop hotspot |

**Calibration is stored in scale_calibration.json: keep it with hardware.py.**

# Test results · EE verification

Measured on 20 October 2026 on the initial stand. Repeat the accuracy test after the final stand is built.

## R1 · Weight accuracy ≤ ±5 g · PASS

| Weights (g) | Expected (g) | Reading (g) | Error (g) |
|---|---|---|---|
| nothing | 0 | 0.1 | ±0.1 |
| 10 | 10 | 10.0 | ±0.1 |
| 20 + 10 | 30 | 30.0 | ±0.1 |
| 50 | 50 | 50.0 | ±0.1 |
| 100 | 100 | 100.1 | +0.1 |
| 100 + 50 | 150 | 150.1 | +0.1 |
| all (200) | 200 | 200.3 | +0.3 |

Calibration: 200 g reference set. Largest error 0.3 g (0.15 %). Noise ±0.1 g.

## R2 · Real-time acquisition, no data loss · PASS

| Measure (60 s, 200 g load) | Result |
|---|---|
| Readings acquired (raw) | 622 |
| Readings processed (calibrated + filtered) | 622 |
| Lost between acquisition and processing | 0 |
| Sequence errors | 0 |
| Readings per second | 10.35 average (7–12 in any 1 s) |
| Processing delay | 0.09 ms average, 0.42 ms max |
| Noise (std dev) | 0.08 g |

The one second with 7 readings is an HX711 restart (sensor timing), not a processing loss.

## Other checks

| Part | Test | Result | Status |
|---|---|---|---|
| Screen | desktop on DSI, touch (Goodix at 0x5d) | works, no setup needed for touch | PASS |
| Camera | detected · photo · focus at plate distance | imx708, 12 MP, weights sharp on the plate | PASS |
| Power | `vcgencmd get_throttled`, all parts connected | `throttled=0x0` | PASS |
| Battery | full app running on battery: runtime + low-voltage | after the app is ready (EE step 5) | pending |

**Battery runtime will be measured with the finished app and compared with the 2.8 h (full load) prediction.**